00 Pattern Selector & Multi-Pattern Cases

read first · then Notes 01–10

Which pattern? — decision map

CREATIONAL
object creation

- Only ONE (or exactly N) instance shared by everyone? -> SINGLETON (Multiton)
- Create one of several TYPES chosen at runtime, hide "new"? -> FACTORY METHOD
- Signals: "only one", "single global", "same file", "unique ID" | "different types of X depending on user request", "too many if-else for creation"

STRUCTURAL
composition / wrapping

- Interface INCOMPATIBLE (legacy / vendor) -> ADAPTER
- Many subsystems, want ONE simple entry -> FACADE
- SAME interface, CONTROL access / lazy / cache -> PROXY
- SAME interface, ADD optional features -> DECORATOR
- Signals: "cannot modify", "different method signatures" | "one interface for inventory, payment..." | "restricted / load only a few pages / permissions" | "add-ons without subclasses for every combination"

BEHAVIOURAL
responsibility / communication

- SAME steps in fixed order, some steps differ per type -> TEMPLATE METHOD
- Request passes LEVELS / checks until one handles (or all pass) -> CHAIN OF RESPONSIBILITY
- Object moves through STATES; same action behaves differently -> STATE
- Signals: "follow the same overall process", "however X is different" | "levels of support / approval limits / sequence of checks, easy to extend" | "can be in states", "transitions to", "draw a state diagram"

All patterns on one table

Pattern	One-line intent	Trigger phrases in a case study	UML shape to draw	Note
Singleton C	Only one instance + global access	only one / single global / same file / unique ID / run by one user / new object every time ⇒ duplicates	1 class: `-instance`, private ctor, `+getInstance()`	01
Multiton C	Exactly N instances	max 5 batsmen / 3 printers / N gates	static List/Map capped at N	01
Factory Method C	Subclass/factory decides which class to instantiate	create different types of X depending on request; hide `new`; extensible product framework	Product interface + concretes; Factory with `create(type)` (or abstract Creator)	02
Facade S	One simple interface to a complex subsystem	one simple interface by internally calling A, B, C; combine air/hotel/taxi; hide complexity	Client → Facade → many subsystem classes	03
Adapter S	Convert an interface into the one the client expects	legacy / vendor / can't modify; new device has a different method; third-party web services with different signatures	Client → Target«i» ← Adapter ◆→ Adaptee	04
Proxy S	Placeholder that controls access to the real object	should not access directly; permissions; load only a few pages; lazy; cache; remote; log access	Subject«i» ← Proxy → RealSubject (both implement Subject)	05
Decorator S	Add responsibilities dynamically by wrapping	optional add-ons/toppings; price = base + selections; without subclasses for every combination	Component«i» ← Concrete + abstract Decorator (has-a Component) ← concrete decorators	06
Template Method B	Fixed algorithm skeleton, subclasses fill steps	same overall process / common workflow; however step X differs; enforce the procedure	abstract class with `final templateMethod()` + abstract steps; subclasses	07
Chain of Resp. B	Pass request along handlers until handled	levels of support; approval limits; sequential checks easy to extend; sub-filters; priorities; exceptions	abstract Handler with `next` self-link; concrete handlers	08
State B	Behaviour changes with internal state	moves through states; transitions; behaviour depends on current state; draw state diagram	Context ◇→ State«i» ← concrete states (+ state diagram)	09

Code smell (case-study lecture)	Pattern
Too many if-else for object creation	Factory
Bloated class with optional features	Decorator
Complex subsystem calls scattered everywhere	Facade
Behaviour changes via flags/conditions	State
Multiple handlers hard-coded in sequence	CoR
Copy-pasted algorithm with small differences	Template Method
Several `new Manager()` for one resource	Singleton

Often confused	Difference in one line
Adapter vs Facade	Adapter **converts** one interface; Facade **simplifies** many
Proxy vs Decorator	both same interface: Proxy **controls access**; Decorator **adds features**, stackable
Proxy vs Adapter	Proxy keeps the **same** interface; Adapter **changes** it
State vs CoR	State: "how do I behave now?"; CoR: "who handles this request?"
Template vs Strategy	Template: inheritance, varies steps; Strategy: composition, swaps whole algorithm
Factory vs Facade	Factory **creates**; Facade **coordinates** existing objects
Singleton vs Factory	Singleton controls **how many**; Factory controls **which type**

How to answer a 25-mark case study

1. **Understand the problem first** (case-study lecture): what varies? what must stay fixed? what will change in future?
2. **Categorize:** creation / structure / behaviour → decision map.
3. **Name the pattern + category + GoF intent** (1 line).
4. **Map the case to participants** ("Target = Printer, Adapter = …") — this is where marks are.
5. **Reasons** (2–4): problem removed + benefit (coupling, OCP, reuse, consistency).
6. **Diagram:** scenario diagram (boxes/arrows) if asked "depict the scenario"; class diagram with stereotypes, +/−/#, underline static, correct arrows.
7. **Code:** interface → concrete → pattern class → 3-line client.
8. **Challenges → mechanism** pairs (table in each note).
9. **State assumptions** (paper says so).

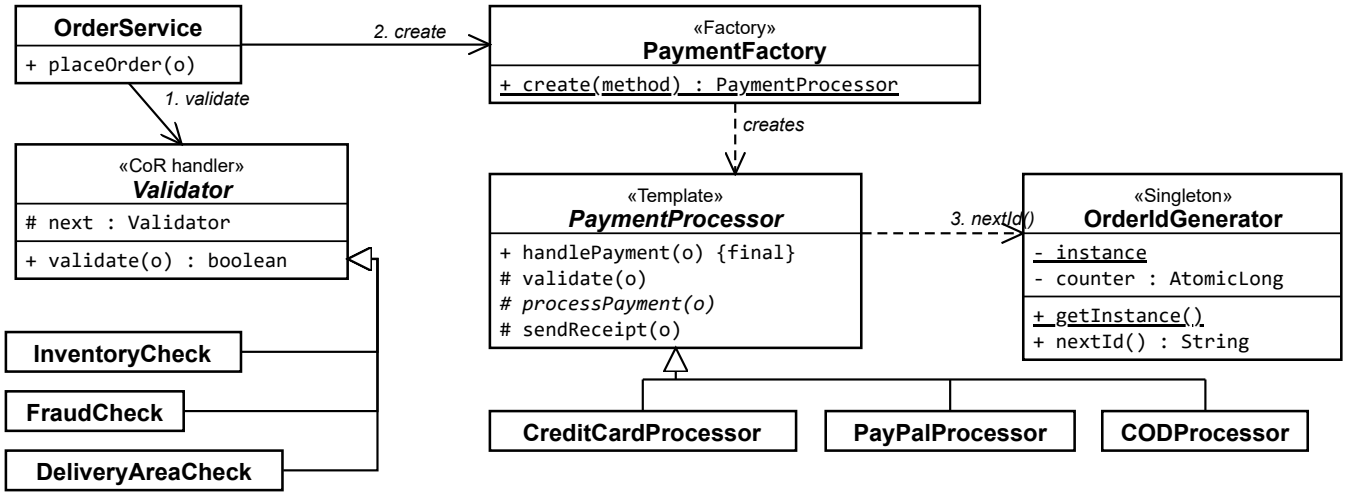
UML notation used in all notes	
«interface» / *Italic*	interface / abstract class or method
`+ - #`, underline	public, private, protected; static
solid line + hollow △	extends (inheritance, is-a)
dashed line + hollow △	implements (realization)
solid line + open arrow	association (has a reference)
◆ filled / ◇ hollow diamond	composition (owns) / aggregation (wraps)
dashed line + open arrow	dependency (uses, creates)
rounded box + filled arrow, ●	state + transition (event label), initial state

Multi-pattern questions — every combination seen so far

Case	Part → pattern	How they fit together
Smart parking *(Quiz 1)*	session states → **State**; vendor payment machine → **Adapter**	`PaymentDueState.pay()` calls `PaymentProcessor` = `LegacyPaymentAdapter` wrapping the vendor class (N09, N04).
Database objects *(Quiz 2)*	MySQL/PostgreSQL/Mongo → **Factory**; one active connection → **Singleton**	`DatabaseFactory.getDatabase(type)` creates the type; `DatabaseConnectionManager.getInstance()` keeps one connection (N02, N01).
Library scanner *(Quiz 3)*	new scanner → **Adapter**; one borrowing interface → **Facade**	`LibraryFacade.borrowBook()` calls login, inventory (uses `ScannerAdapter`), payment, notification (N04, N03).
Printing *(Quiz 4)*	one PrinterManager → **Singleton** (4 printers → Multiton)	N01.
User profile *(Quiz 5)*	badge, 2FA, theme → **Decorator**	N06.
Course completion *(Quiz 6)*	fixed completion flow → **Template Method**; optional AI/premium/mentor → **Decorator**	`completeCourse()` final in `OnlineCourse`; extras wrap the `Course` features/price (N07, N06).
Tech support *(Quiz 7)*	L1→L2→L3→Specialist → **CoR**; same log→solve→log → **Template Method**	`final handle()` in the abstract handler; levels implement `attemptToSolve()` (N08).
Travel booking *(2022 Q1)*	combine air+hotel+taxi → **Facade**; third-party services → **Adapter**	Facade on top, one adapter per provider below each sub-service (N03, N04).
Payroll *(2022 Q2, 2023 Q1)*	salary per employee type → **Template Method**; run by one user → **Singleton**; changing parameters → Singleton config	N07, N01.
Restaurant *(2023 Q3)*	coffee combos → **Decorator**; vendor event sites → **Adapter**; logging → **Singleton** (or logging Proxy)	N06, N04, N01/N05.
Coffee shop + logger *(2024 Q1)*	**Decorator** + **Singleton** logger	N06, N01.

Workflow document *(2024 Q3)*	**State** + interfaces	N09.
Checkout *(2024 Q4)*	standard steps → **Template**; exceptions → **CoR**; authorization levels → **Proxy / CoR**	N07, N08, N05.
Self-checkout kiosk *(2025 Q1)*	kiosk states → **State**; add-ons → **Decorator**; global tax calculator → **Singleton**	Scanning adds decorated items; Payment state sums `getCost()` and calls `TaxCalculator.getInstance()` (N09, N06, N01).
Library automation *(2025 Q3)*	sequential checks → **CoR**; book types → **Factory**; rare books → **Proxy**; optional services → **Decorator**	Request passes Membership→Availability→Fine chain; `BookFactory` creates the book; `ArchiveAccessProxy` guards rare books; borrow transaction wrapped with SMS/Email/Extended decorators.
E-commerce orders *(2025 Q4)*	payment workflow → **Template**; validations → **CoR**; OrderID → **Singleton**; payment objects → **Factory**	See combined diagram below.
Cake shop / LMS / bank *(2022–23 Q4)*	cake options → Decorator; extensible framework → Factory; LMS pages → Proxy; LMS modules → Factory; accounts → Factory; exceptions by priority → CoR; authorization → Proxy	Short 5-mark answers: name pattern + 2 reasons + tiny class sketch.

Combined design — 2025 Q4 e-commerce (CoR + Factory + Template + Singleton)



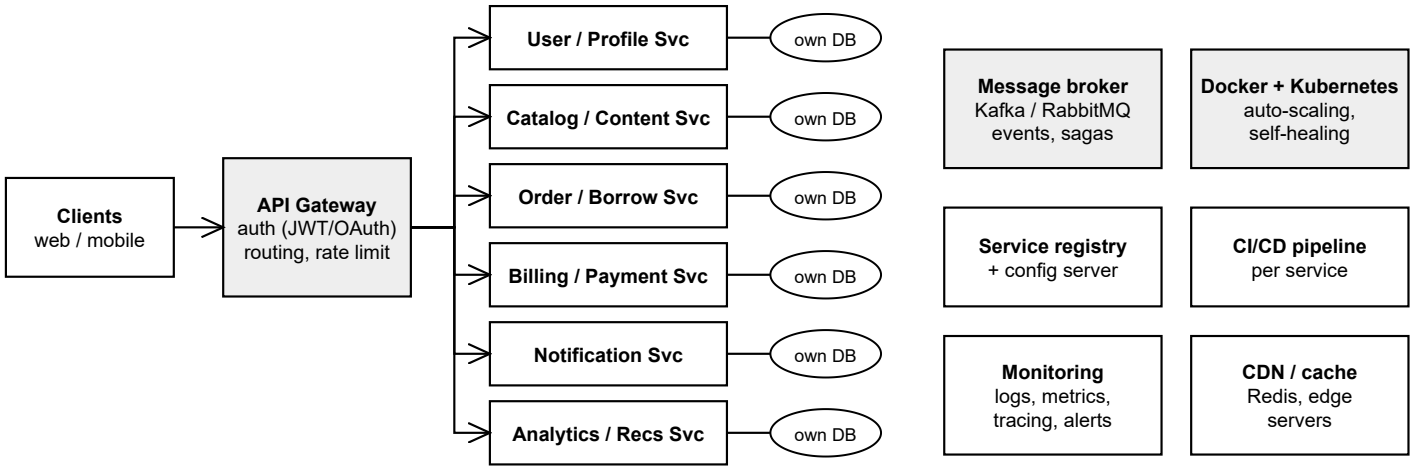

```
public void placeOrder(Order o) {
    Validator chain = new InventoryCheck();                                    // CoR
    chain.linkWith(new FraudCheck()).linkWith(new DeliveryAreaCheck());
    if (!chain.validate(o)) { System.out.println("Order rejected"); return; }
    PaymentProcessor p = PaymentFactory.create(o.getPaymentMethod());          // Factory
    p.handlePayment(o);              // Template: id = OrderIdGenerator.getInstance().nextId() (Singleton) -> validate ->
process -> receipt
}
```

Lecture case studies (likely exam scenarios): **Food delivery** — Facade (order flow), Adapter (restaurant/payment APIs), State (order status), Factory (payment), Singleton (config), Observer-like notifications. **Smart home** — Facade (“away mode”), Adapter (Zigbee/Bluetooth devices), State (device modes), Proxy (remote/authorization). **LMS** — Factory (modules), Proxy (lazy content, access), Template (exam/grading flow), Decorator (premium features). **ATM** — State (idle/card inserted/PIN ok), Singleton (bank connection), Proxy (authorization), CoR (cash dispenser denominations / validations). **Shopping cart** — Decorator (promotions/gift wrap), Template (checkout), Factory (payment), CoR (validations), Singleton (ID/tax). **Travel** — Facade + Adapter. **Video streaming** — Proxy (subscription/protection, CDN cache, lazy load), Decorator (subtitles/HD), Facade (“Play”), State (player states).

Generic answer bank

Why not inheritance for everything?	Class explosion, rigid compile-time hierarchy, changes ripple to subclasses ⇒ prefer composition (Decorator, State, Strategy, Adapter).
Use of interfaces / abstract classes	Interface = contract (CAN-DO), enables polymorphism and loose coupling, DIP, swap implementations, mocking. Abstract class = shared code + enforced steps (IS-A), used by Template Method and abstract Decorator/Handler.
General pattern trade-offs	+ reuse, flexibility, loose coupling, maintainability, common vocabulary. − more classes/abstraction, learning curve, overuse = over-engineering; "Is it worth the extra abstraction?" (case-study lecture).
Configurability (parameters change over time)	Move rates/limits to external config (DB table, properties/YAML) read by a Singleton `ConfigManager`; admin UI to update; version and audit changes; no recompilation.
High Availability (HA) & Disaster Recovery (DR)	**HA:** stateless services replicated behind a load balancer, health checks and auto-restart (Kubernetes), no single point of failure (avoid single Singleton server → replicas), DB replication/cluster, retries/timeouts/circuit breaker, fallback handler in CoR. **DR:** regular backups, replicated DB to a second region/data centre, defined RPO/RTO, automated failover, infrastructure-as-code to rebuild, periodic DR drills.

Q2 in every past paper: Microservices (not in the slides — standard answers)



Each service: own codebase, own DB, own deployment; talk via REST/gRPC (sync) or events (async)

Draw the scenario	Clients → API Gateway → one service per business capability, **each with its own DB**; message broker for events; registry, config, monitoring; containers on Kubernetes. Library: Book, User, Borrowing, Notification, Catalog Search (search index e.g. Elasticsearch). Supermarket chain: Purchasing, Sales/POS, Warehouse/Inventory, Supplier, Branch, Reporting. Streaming: User Profile, Content Delivery (transcode + CDN), Recommendation (ML), Billing/Subscription, Analytics.
Scalability, fault isolation, continuous deployment	**Scalability:** each service scales independently (horizontal pods, auto-scaling on CPU/requests; e.g. scale Content Delivery and Search, not Billing); stateless services; caching/CDN. **Fault isolation:** separate processes/DBs; circuit breakers, timeouts, bulkheads, retries with back-off; graceful degradation (recommendations down → show popular list). **CD:** own repo + CI/CD pipeline per service, automated tests, Docker images, blue-green/canary releases, versioned APIs; independent teams per service; centralized logging, metrics and tracing.
Data consistency	Database-per-service ⇒ no shared transactions; use **Saga pattern** (sequence of local transactions with **compensating actions**, choreography via events or orchestration), **eventual consistency** via event-driven messaging (Kafka), **transactional outbox**, idempotent consumers, CQRS for read models (catalog search), two-phase commit only where strong consistency is essential (rare), data versioning.
Security	API Gateway as single entry: **authentication** (OAuth2/OpenID Connect, JWT tokens), **authorization** by role (RBAC); **service-to-service**: mutual TLS, service mesh, short-lived tokens; HTTPS everywhere; encrypt sensitive data at rest (PCI-DSS for payments); secrets vault; rate limiting, input validation, WAF; least privilege per service DB; audit logging and monitoring.
Same hardware/config for each platform? *(2023)*	Consistent environments (containers/IaC) make deployment, scaling and failover predictable: any node can run any service, same performance baseline, easier capacity planning, automation, monitoring and troubleshooting; avoids "works on my machine"; simplifies DR (rebuild identical nodes). Exceptions: GPU nodes for ML recommendation services.